CSE440, Section 01 | Group 7 | Kazi Eraj Al Minahi Turjo (1831906642), Md. Sabbir Hossain (2212642042), Nashita Tasneem Noor (2132126642), Talha Imtiaz (2012211642)

Strategic Poker Game Simulation Using Expectiminimax Under Uncertain Card Events

Kazi Eraj Al Minahi Turjo
Department of ECE
North South University
Dhaka, Bangladesh
kazi.turjo@northsouth.edu

Md. Sabbir Hossain
Department of ECE
North South University
Dhaka, Bangladesh
sabbir.hossain123@northsouth.edu

Nashita Tasneem Noor
Department of ECE
North South University
Dhaka, Bangladesh
nashita.tasneem@northsouth.edu

Talha Imtiaz
Department of ECE, North South University, Dhaka, Bangladesh
talha.imtiaz@northsouth.edu

Abstract

This report presents the design and implementation of a simplified Texas Hold'em poker simulator with an Expectiminimax-based artificial intelligence agent for decision-making under uncertainty. Poker is a suitable artificial intelligence problem because the agent must act with hidden opponent cards, stochastic future community cards, and incomplete knowledge of the opponent's intention. The implemented system is written in Python and organized into independent modules for card and deck management, player state tracking, hand evaluation, game flow, Expectiminimax search, user interaction, and simulation-based evaluation. A Streamlit interface is used to make the simulator demonstrable through a poker table view, beginner-oriented guidance, and repeated-hand evaluation tools. The AI decision module represents the agent's decision as MAX nodes, the opponent response as a simplified MIN/policy node, and unknown cards as CHANCE nodes approximated using Monte Carlo sampling. Experimental runs over 100 automated hands show that the Expectiminimax mode achieved a 57.0% positive-hand rate with a total chip change of +860 and an average decision time of 0.152 seconds per hand in the tested configuration. The results indicate that the simulator successfully connects core game theory concepts with a practical, interactive poker environment suitable for a course-level artificial intelligence project.

Index Terms— Poker AI, Expectiminimax, Game Theory, Monte Carlo Simulation, Imperfect Information, Streamlit, Texas Hold'em, Decision Making.

1 Introduction

Poker is a challenging domain for artificial intelligence because a player never sees the complete state of the game. In Texas Hold'em, each player has two private cards, while the board is gradually revealed through the flop, turn, and river. Therefore, unlike deterministic games such as tic-tac-toe or chess, a poker agent cannot simply search one complete known tree. It must reason about unknown opponent hole cards, possible future community cards, betting decisions, and the expected value of continuing versus folding.

This project develops a simplified Texas Hold'em poker simulator that applies Expectiminimax to this uncertain setting. The main goal is not to build a professional poker engine, but to demonstrate how an AI agent can choose actions when the game contains both adversarial and random elements. In our formulation, the AI's own choices are treated as MAX decisions, opponent reactions are approximated using a simple policy/minimizing layer, and future card events are treated as chance outcomes. This makes poker a practical example of the game-tree concepts studied in CSE440, especially minimax, stochastic games, and utility-based decision-making. This final report represents the work of CSE440 Section 01, Group 7.

The project also focuses on usability. Instead of only providing command-line output, the final implementation includes a Streamlit-based web interface that displays the poker table, players, stacks, community cards, pot information, a beginner help panel, and simulation/evaluation results. A separate kid-friendly play layer was also included to make the basic poker flow understandable to new users.

The major contributions of this work are:

- a modular Python implementation of a simplified Texas Hold'em simulator;
- a hand evaluation module that compares the best five-card hand from available cards;
- an Expectiminimax decision module using depth limits and Monte Carlo sampling;
- an interactive Streamlit interface for demonstration and beginner explanation;
- an evaluation framework for comparing Expectiminimax play against a baseline strategy.

1.1 Motivation

The project topic is important because many real decision problems resemble poker. In business negotiation, bidding, cybersecurity, finance, and resource allocation, an agent rarely knows the full intention or future move of other parties. Poker gives a small but meaningful environment to practice strategic reasoning under such uncertainty. The simulator therefore acts both as a playable demonstration and as an educational platform for understanding how probabilistic decision trees operate.

1.2 Relation to Prior Work

Research in poker AI has produced strong systems using abstraction, self-play, regret minimization, and equilibrium-based techniques. Counterfactual regret minimization has been influential in imperfect-information games, and systems such as Libratus and Pluribus demonstrated superhuman poker performance in more advanced settings [1, 4, 5]. Our project is smaller in scope. It focuses on a course-level, transparent implementation where each module can be explained, tested, and presented clearly. This is why Expectiminimax with sampling was selected instead of a more complex professional-level poker solver.

2 Objectives and Scope

The simulator was designed with five practical objectives. First, the system should model the essential rules of Texas Hold'em: deck generation, shuffling, dealing hole cards, revealing community cards, betting stages, pot updates, and showdown resolution. Second, the hand evaluator should correctly rank common poker categories such as pair, two pair, straight, flush, full house, and four-of-a-kind. Third, the AI should use an Expectiminimax-based decision structure rather than a purely random or fixed rule strategy. Fourth, the interface should allow a user to observe the game and understand why the AI chooses an action. Finally, the project repository should follow the expected course structure with a main file, README, requirements file, data folder, support folder, and other submission materials.

The scope is intentionally simplified. The simulator currently focuses on two-player heads-up play for reliable demonstration. Side pots, professional betting abstractions, multi-table tournaments, and large-scale self-play training were kept outside the final scope. These decisions kept the implementation manageable while preserving the important artificial intelligence ideas.

3 System Architecture

The system is organized as a layered Python application. Figure 1 shows the overall architecture. The top layer contains the Streamlit interface, where the user can open the game, view the beginner mode, and run simulations. The middle layer contains the game engine, player objects, card/deck utilities, hand evaluator, and metric collection. The bottom decision layer contains the Expectiminimax AI.

3.1 Game Logic Layer

The game logic layer is mainly responsible for running a complete poker hand. It starts a new hand, initializes a shuffled deck, deals two hole cards to every player, posts small and big blinds, moves through betting rounds, reveals community cards, and resolves the hand at showdown. The game stage is represented using labels such as pre-flop, flop, turn, river, and showdown.

The core game state can be described as

$$S_t = (P_t, C_t, B_t, R_t, D_t), \tag{1}$$

where P_t represents the player states, C_t represents community cards, B_t stores betting and pot information, R_t indicates the current round, and D_t represents dealer/blind position. This state representation is sufficient for the simplified simulator and is also transformed into a smaller search state when the AI needs to evaluate an action. Table 1 summarizes how these state variables map into implementation-level data.

3.2 Card, Deck, and Player Representation

Cards are represented by rank and suit, while the deck is a collection of standard playing cards that is shuffled before dealing. The player object stores name, stack, current bet, hole cards, folded status, and all-in status. This separation makes the code easier to debug because card handling, player state, and game flow are not mixed together.

3.3 Hand Evaluation Module

The hand evaluator receives the available cards and returns a comparable hand score. In Texas Hold'em, a player can form the best five-card hand from seven cards: two private hole cards and five community cards. The evaluator therefore checks all five-card combinations and selects the best ranking. The ranking categories are encoded from high card up to straight flush. This approach is computationally acceptable for the project size because there are only $\binom{7}{5} = 21$ five-card combinations for a complete seven-card hand.

Table 1: Game State Components Used by the Simulator

Symbol	Meaning in this project
P_t	Player objects with stack, hole cards, current bet, folded flag, and all-in flag.
C_t	Public board cards visible to all active players.
B_t	Pot size, blind amounts, and current betting information.
R_t	Current street: pre-flop, flop, turn, river, or showdown.
D_t	Dealer position used for blind posting and turn order.

4 User Interface and Repository Structure

The project uses Streamlit to provide a browser-based interface. The main application contains three tabs: Play Game, Kid Play, and Simulation & Evaluation. The Play Game tab displays the table state and lets the user run a hand while changing the AI depth and sample count. The Kid Play tab explains the game in simpler wording. The Simulation & Evaluation tab runs repeated automated hands and shows the resulting metrics.

The repository was organized for course submission. The main file is `main.py`; core logic is placed under the `poker_ai` package; support files and report/presentation files are separated into their own folders. Figure 4 summarizes the major implementation files and their responsibilities.

4.1 Interface Walkthrough

A typical demonstration starts from the Play Game tab. The presenter first shows the current table, identifies Seat 0 as the AI player, and explains the private cards, community cards, pot, and player stacks. Then the search depth and Monte Carlo sample sliders are adjusted to show that the AI is configurable. After pressing the play button, the system completes a hand and records the winner. The right-side help panel is useful during presentation because it explains the meaning of fold, call, check, and raise without requiring the audience to know poker beforehand.

The Simulation & Evaluation tab is used after the single-hand demo. It runs many hands automatically and outputs the win rate, loss rate, average profit, and average decision time. This separation is important: one hand is good for explaining the interface, while many hands are needed to discuss whether the AI behavior is stable. The Kid Play tab provides a simplified learning layer so that users can follow the game even if they are unfamiliar with poker vocabulary.

5 Expectiminimax Algorithm Implementation

The central AI idea in this project is Expectiminimax. Standard minimax assumes that the game tree has only the agent and the

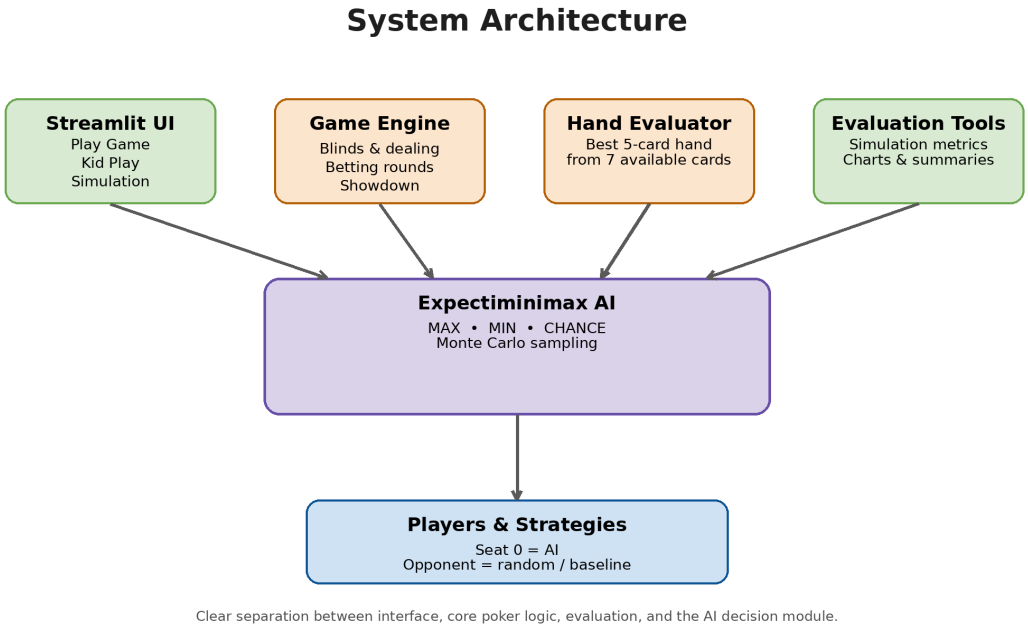

Figure 1: Overall system architecture showing the separation between the Streamlit interface, poker game logic, evaluation tools, and Expectiminimax AI module.

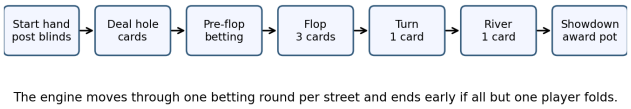

Figure 2: Simplified Texas Hold'em hand progression implemented in the game engine.

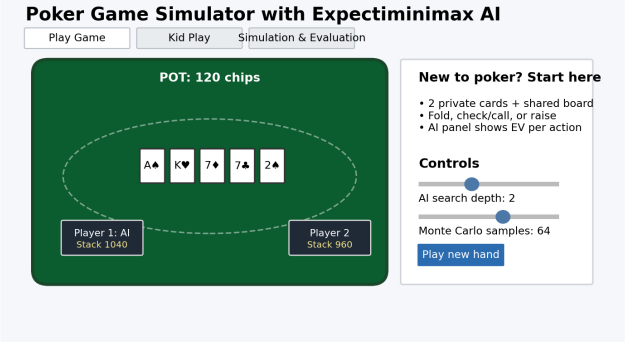

Figure 3: Representative Streamlit interface layout based on the implemented UI: poker table view, help panel, controls, and game status.

opponent. Poker also includes random card events, so a third type of node is required. Expectiminimax extends minimax by adding chance nodes, where the value is an expectation over possible random outcomes [6].

5.1 Decision Node Types

The search tree has three main node types:

- **MAX node:** the AI chooses the action with the highest estimated value;
- **MIN/opponent node:** the opponent response is approximated through a simple policy;

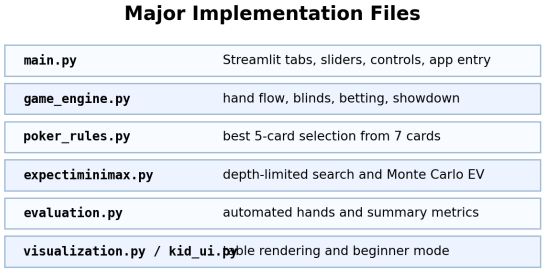

Figure 4: Major implementation files used in the final project.

- **CHANCE node:** unknown cards are sampled from the remaining deck.

The value function can be written as

$$V(s) = \begin{cases} \max_{a \in A(s)} V(T(s,a)), & \text{AI turn,} \\ \min_{a \in A(s)} V(T(s,a)), & \text{opponent turn,} \\ \sum_{o \in O(s)} P(o|s) V(T(s,o)), & \text{chance event.} \end{cases} \tag{2}$$

Here, $A(s)$ is the set of legal actions, T is the transition function, and $O(s)$ is the set of possible chance outcomes. In the actual project, exact enumeration of all future card combinations is replaced by sampling to keep response time reasonable.

5.2 Action Selection

For each decision, the AI receives a simplified search state containing the current board, the AI hole cards, pot size, stack sizes, player to move, and game stage. Legal actions include fold, call/check, and raise. To reduce branching, raise actions are discretized into a few possible raise sizes instead of every possible chip amount. The selected action is the one with the highest estimated expected value at the root node.

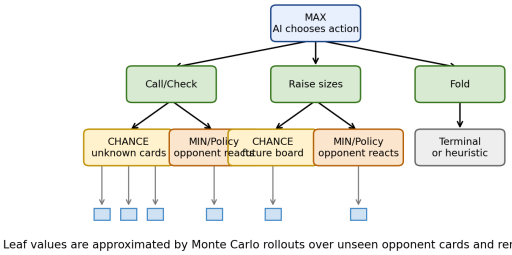

Figure 5: Expectiminimax tree structure used by the poker AI. MAX nodes represent AI decisions, policy/MIN nodes represent opponent reaction, and CHANCE nodes represent unknown card events.

Table 2: Monte Carlo Hand-Strength Evaluation Procedure

Algorithm: Approximate value of a poker state
1. Remove known AI cards and community cards from deck.
2. Repeat for n samples.
3. Draw possible opponent hole cards.
4. Draw missing board cards until the board has five cards.
5. Evaluate AI and opponent seven-card hands.
6. Add $+pot$ for AI win, $-pot$ for AI loss, and 0 for tie.
7. Return average sampled value.

5.3 Search Pseudocode

The root-level decision procedure can be summarized as follows. The algorithm creates a small list of possible actions, evaluates each action using the recursive search and sampling procedure, stores the root analysis for visualization, and returns the action with the highest estimated value.

Listing 1: Simplified root decision logic used by the AI

```
best_ev = -infinity
for action in candidate_actions:
   next_state = apply_ai_action(state, action)
   ev = eval_state(next_state, depth - 1)
   store_root_analysis(action, ev)
   if ev > best_ev:
      best_ev = ev
      best_action = action
return best_action
```

This structure keeps the code readable for presentation. It also makes the AI decision panel possible because every root action and its estimated value are saved before the final action is returned.

5.4 Complexity Consideration

The exact poker search tree is too large for a small course project because every unknown card combination and every possible bet size can create new branches. The implementation reduces this complexity in three ways. First, the search depth is limited and controlled from the interface. Second, raise sizes are discretized into small, medium, and all-in style candidates. Third, chance outcomes are sampled instead of fully enumerated. These simplifications trade perfect optimality for a practical response time and a transparent algorithm.

5.5 Monte Carlo Evaluation

Exact poker tree search is expensive because the AI does not know the opponent's cards and may also need to consider missing board cards. Therefore, the heuristic evaluation uses repeated random samples from the remaining deck. For each sample, the evaluator draws possible opponent hole cards and completes missing community cards. The AI hand and opponent hand are then compared. The average result over the samples becomes an approximate expected value.

5.6 Opponent Approximation

A complete poker opponent model would require learning from behavior over many hands. For this project, a compact opponent policy was used to keep the model explainable. The opponent continues when the pot is attractive and folds otherwise. This is not a perfect human model, but it provides a stable adversarial response layer that allows the AI to evaluate the consequences of raising, calling, or folding.

6 Implementation Details

6.1 Game Engine

The game engine supports two to four players, but the final demonstration focuses on two-player heads-up poker. At the start of each hand, player states are reset, the deck is shuffled, and hole cards are dealt. The blind posting logic is adjusted for heads-up play, where the button posts the small blind and the other seat posts the big blind. Each street then runs a betting round and updates the pot.

A hand may end early if all but one player folds. Otherwise, the game reaches showdown, evaluates each active player's best hand, and distributes the pot. The last-hand summary is saved so that the interface can show the winner, hand category, final pot, and player states.

6.2 Hand Ranking

The hand evaluator works by checking every possible five-card combination from the available cards. Each five-card hand is classified by category and tie-breaker values. Since the evaluator returns tuple-like comparable scores, comparing hands becomes straightforward in Python. This design avoids many special-case comparison bugs and supports readable testing.

6.3 Visualization and Beginner Mode

The visualization module injects custom CSS and renders a poker-table styled layout. It displays cards, player states, pot information, and the last-hand report. A beginner help section explains the meaning of the current stage and the purpose of each action. The Kid Play tab builds on this idea by adding simpler language for users who are not familiar with poker.

6.4 Evaluation Module

The evaluation module runs many automated hands and collects values such as hand number, chip delta, current stacks, winner, mode, and decision time. These values are stored in a tabular format and summarized into win rate, loss rate, average profit, and average decision time. This makes the project easier to demonstrate because the AI can be tested over multiple hands instead of relying on one lucky or unlucky example.

7 Experimental Evaluation

7.1 Evaluation Setup

The final evaluation compared two modes. In the first mode, Seat 0 used the Expectiminimax AI with depth 2 and 64 Monte Carlo samples. In the second mode, Seat 0 used a simpler

Table 3: Evaluation Metrics Recorded During Simulation

Metric	Definition / purpose
AI chip delta	Chip change of the AI player in a single hand. Positive means the AI gained chips; negative means it lost chips; zero means no net change.
Cumulative chip change	Running sum of AI chip deltas across all simulated hands. This shows the overall trend over the evaluation run.
Positive-hand rate	Percentage of hands where AI chip delta is greater than zero. Ties/no-change hands are not counted as positive.
Loss/tie rate	Percentage of hands where AI chip delta is negative or exactly zero, used to separate losses from no-change outcomes.
Decision time	Wall-clock time needed to complete a hand decision process in the selected mode.
Winner label	Recorded winner name(s), used to confirm whether the hand ended by fold or showdown.

baseline that mainly checks or calls when possible. Seat 1 used a random strategy opponent. Each mode was tested for 100 automated hands using the same project evaluation framework. Table 3 lists the main metrics used in the report.

7.2 Results

Figure 6 summarizes the automated evaluation. In the tested run, the Expectiminimax mode achieved a positive chip change in 57.0% of hands, a loss in 26.0% of hands, and no change/tie in the remaining hands. The total chip change for the Expectiminimax mode was +860 over 100 hands, with an average profit of +8.6 chips per hand. The average decision time was 0.152 seconds per hand.

Here, positive-hand rate means the proportion of hands for which the AI chip delta is greater than zero. Hands with zero chip change are treated as no-change/tie outcomes and are not counted as positive hands. Chip profit, however, depends not only on how many hands are positive but also on the size of each win and loss. Therefore, a mode can have a lower positive-hand rate but still show a higher cumulative chip gain in a short stochastic run if its few wins are larger or its losses are smaller.

The baseline produced a positive chip change in 42.0% of hands and a total chip change of +990 in the tested run, but its decision time was much lower because it did not run a search process. This pairing should not be interpreted as a final claim that the baseline is strategically stronger. It shows the variance of a 100-hand poker simulation and the difference between frequency-based and chip-magnitude-based metrics. The main conclusion is that the Expectiminimax mode provides explainable search-based decisions and a higher positive-hand frequency in this representative run, while the baseline is faster and can still produce favorable chip swings due to poker variance.

7.3 Root Decision Example

Figure 8 shows one example of AI root decision analysis. For one state, the AI compared check and multiple raise sizes. The largest raise had the highest estimated EV in that sample, so the AI selected it. This does not prove that large raises are always optimal; instead, it demonstrates that the action is chosen by comparing estimated future values rather than by a fixed if-else rule.

7.4 Interpretation

The results are best interpreted as a functional course-level validation rather than a professional poker benchmark. The project successfully demonstrates that the simulator can run repeated hands, collect metrics, and show how AI search changes decision behavior. The win rate and chip profit vary because poker is stochastic and the opponent behavior is random. A larger number of runs, fixed random seeds, and more advanced baselines would be needed for stronger statistical claims.

7.5 Threats to Validity

There are several reasons why the numbers should be treated carefully. First, a 100-hand run is small for poker, where variance can be high. Second, the chip stack does not represent tournament-level poker strategy; it is mainly used as a measurable utility value. Third, the baseline is simple, so beating or comparing with it does not prove superhuman ability. Fourth, the random opponent can sometimes produce unusual outcomes because it does not behave like a trained human. For these reasons, the evaluation is presented as evidence of functionality and strategy integration, not as a final poker-AI research claim.

7.6 Parameter Sensitivity

The two most important AI parameters are search depth and sample count. Increasing depth allows the AI to look further into the future, but it also increases computation. Increasing samples improves the stability of the expected value estimate, but it also increases decision time. In the final configuration, depth 2 and 64 samples were selected as a balanced setting for classroom demonstration. This setting gives visible decision analysis without making the interface feel too slow.

8 Rules Compliance and Testing

Testing focused on the core mechanics needed for a correct simplified Texas Hold'em simulator. The deck contains standard card identities and draws are performed without duplication. Each player receives exactly two hole cards. Community cards progress through zero cards before the flop, three after the flop, four after the turn, and five after the river. The game supports early termination when a player folds and showdown evaluation when multiple active players remain.

Scenario-based testing was also performed during development. Examples included early folds, all players calling, multiple raises, strong shared boards, flush-versus-straight comparisons, and full house versus four-of-a-kind comparisons. These tests were useful because many poker bugs do not appear in a single random run. Testing with known hands makes it easier to verify whether the evaluator and printed descriptions match the actual cards.

9 Discussion

9.1 Technical Achievements

The strongest achievement of the project is that it connects theory and implementation in one demonstrable system. The game engine represents the environment, the evaluator converts cards into utility-relevant values, and the Expectiminimax

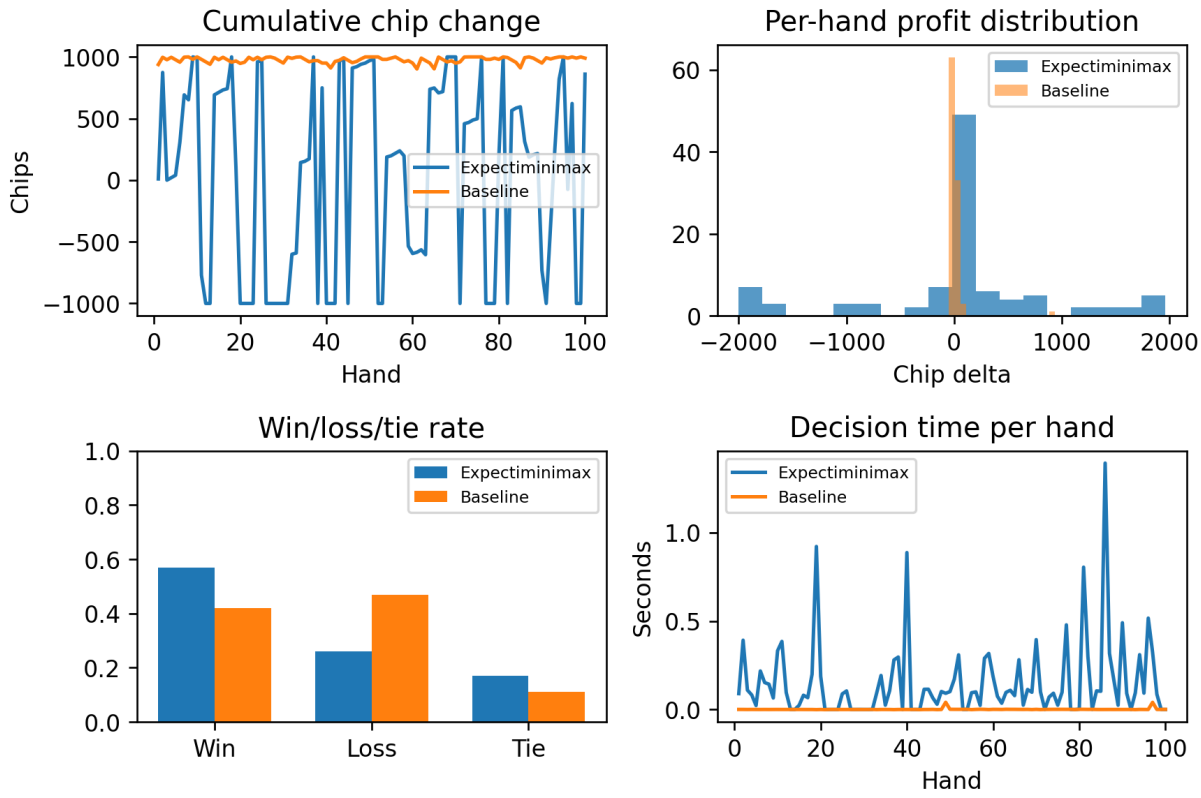

Figure 6: Experimental analysis over 100 automated hands. The figure compares cumulative chip change, per-hand profit distribution, win/loss/tie rate, and decision time for Expectiminimax and baseline modes.

Mode	Expectiminimax	Baseline
Hands tested	100	100
Win rate	57.0%	42.0%
Total chip change	+860	+990
Avg. profit/hand	+8.6	+9.9
Avg. time/hand	0.152s	0.002s

Figure 7: Summary of the automated evaluation metrics for the final simulation run.

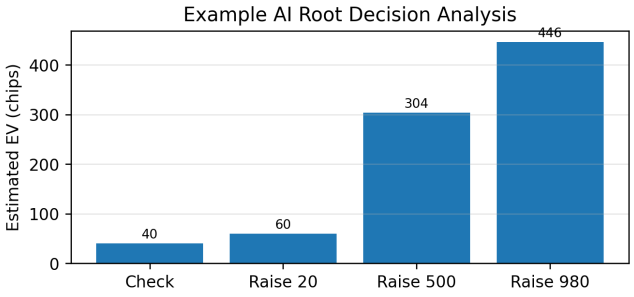

Figure 8: Example root-level action comparison from the Expectiminimax AI.

module uses those values to choose actions under uncertainty. The interface then makes the result visible to users through cards, chips, game stage, and AI decision analysis.

Another achievement is modularity. Because the modules are separated, the game engine can be tested without the UI, the evaluator can be tested with example hands, and the AI can be tuned through depth and sample sliders. This makes the repository easier to explain during presentation and easier to improve later.

9.2 Limitations

The project has several limitations. First, the opponent model is simple and does not learn from long-term behavior. Second, raise sizes are discretized, so the AI does not search every legal bet amount. Third, the evaluation results are influenced by stochastic card draws and random opponent choices, so a single 100-hand run should not be treated as a final scientific benchmark. Fourth, side pots, complex all-in interactions, and professional no-limit poker abstractions are outside the current implementation.

9.3 Future Work

Future work can improve both the AI and the user experience. The opponent model could be upgraded to track aggression, fold tendency, and bet sizing over time. More simulations with fixed random seeds could provide stronger statistical evaluation. The AI could also be compared against multiple baseline strategies such as always-call, tight-aggressive, loose-passive, and random-raise opponents. On the interface side, a more interactive human turn system and better visual animation could make the simulator feel closer to a real poker game.

10 Development Process

The project was developed in stages so that the group could move from planning to implementation and then to evaluation. During the first stage, the topic was finalized and the team reviewed Texas Hold'em rules, minimax, and Expectiminimax. During the second stage, the architecture was divided into game engine, hand evaluator, AI search, testing, and documentation components. During the third stage, the early simulator

Table 4: Functional Testing Summary

Feature Checked	Status
Deck generation and no duplicate card draw	Verified
Two hole cards per player	Verified
Flop, turn, river progression	Verified
Fold and showdown termination	Verified
Poker hand category ranking	Verified with examples
Simulation metric collection	Verified
Streamlit interface rendering logic	Implemented

Table 5: Summary of Development Stages

Stage	Main work completed
Week 1	Topic finalization, course requirement review, algorithm research.
Week 2	Architecture design, rule specification, module planning.
Week 3	Deck, dealing, game loop, pot update, early prototype.
Week 4	Hand ranking, scoring framework, known-hand testing.
Week 5+	Expectiminimax integration, UI polishing, simulation and report preparation.

was implemented and tested using text-based runs. During the fourth stage, the hand evaluator and scoring framework were refined. During the final stage, the Expectiminimax logic, Streamlit interface, and simulation tools were combined into one demonstrable application.

This staged process also helped divide responsibilities. One member could focus on the engine, another on research and algorithm explanation, another on interface clarity and testing, and another on utilities, debugging, and repeated evaluation. The final report combines these parts into a single system description.

11 Applications Beyond the Simulator

Although the implementation is a poker game, the reasoning pattern is not limited to games. Expectiminimax represents a general method for choosing actions when an intelligent agent faces both an opponent and random events. This is why the same idea can be connected to several real-world domains. In financial trading, a decision maker does not know all market intentions and must act under uncertain future movement. In cybersecurity, a defender must choose responses while the attacker's plan is hidden and future attack paths are uncertain. In auctions and bidding, participants estimate the possible actions of other bidders while also reacting to market conditions. Poker is therefore a compact environment for demonstrating a broader class of strategic decision problems.

The project also has educational value. A student can first observe the poker table, then inspect the AI root decision values, and finally run repeated simulations to see how individual decisions affect long-run performance. This makes abstract topics such as utility, expected value, search depth, chance nodes, and heuristic evaluation easier to understand. Instead of only seeing equations, the user can connect each concept to a visible decision in the game.

11.1 Why Expectiminimax Was Appropriate

A simpler random strategy would be easy to implement, but it would not demonstrate strategic reasoning. A pure minimax strategy would also be incomplete because it cannot naturally represent the random arrival of unseen cards. Reinforcement learning could be another option, but it would require training episodes and more time to tune. Expectiminimax was selected because it directly matches the structure of poker: the AI chooses an action, the opponent may respond, and nature reveals cards. This makes the algorithm theoretically suitable and still feasible for a semester project.

11.2 Report and Presentation Usefulness

The final system is suitable for presentation because each part can be shown quickly. The presenter can open the repository, explain the main file, show the modules, describe the game flow, run the one-minute demo video, and then use the report figures to explain the algorithm. The diagrams in this report were prepared to support that explanation: the architecture diagram explains the code organization, the game-flow diagram explains the environment, the decision tree explains Expectiminimax, and the result charts explain evaluation.

12 Member Contributions

Kazi Eraj Al Minahi Turjo (1831906642) led the planning and system architecture, implemented the early game engine, worked on the hand evaluation and Expectiminimax integration, and coordinated final report preparation. Md. Sabbir Hossain (2212642042) focused on related-work research, scenario-based testing, probabilistic reasoning explanations, and Expectiminimax documentation. Nashita Tasneem Noor (2132126642) worked on user-oriented requirements, interface wording, scenario testing, and documentation of user interaction flow. Talha Imtiaz (2012211642) contributed to setup, helper utilities, debugging support, score design discussion, repeated test runs, and evaluation preparation.

13 Conclusion

This project successfully developed a simplified Texas Hold'em poker simulator with an Expectiminimax-based AI agent. The system demonstrates the main idea of strategic decision-making under uncertainty by combining MAX decisions, opponent responses, chance events, and Monte Carlo hand evaluation. The Python implementation is modular and includes a Streamlit interface, beginner guidance, and simulation tools for repeated evaluation.

The experimental results show that the Expectiminimax mode can produce positive chip outcomes while keeping average response time within a practical range for demonstration. More importantly, the system makes the AI decision process visible and explainable, which fits the educational objective of the CSE440 project. Although the simulator is simplified compared with professional poker AI systems, it provides a clear and functional demonstration of how game theory and probabilistic search can be applied to an imperfect-information card game.

Acknowledgment

The authors thank the CSE440 course instructor Mohammad Shifat-E-Rabbi (MSRB) for providing project guidance, time-

line instructions, and presentation requirements for CSE440 Section 01, Group 7. We also acknowledge the use of Python, Streamlit, NumPy, Pandas, and Matplotlib for building and evaluating the simulator. AI-assisted drafting and formatting support was used for organizing language, diagrams, and document polishing; the project code, testing, and final responsibility remain with the group members.

References

[1] M. Bowling, N. Burch, M. Johanson, and O. Tammelin, "Heads-up limit hold'em poker is solved," *Science*, vol. 347, no. 6218, pp. 145–149, 2015.

[2] A. Gilpin and T. Sandholm, "A competitive Texas Hold'em poker player via automated abstraction and real-time equilibrium computation," in *Proc. AAAI*, 2006, pp. 1007–1013.

[3] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret minimization in games with incomplete information," in *Advances in Neural Information Processing Systems*, 2007, pp. 1729–1736.

[4] N. Brown and T. Sandholm, "Superhuman AI for heads-up no-limit poker: Libratus beats top professionals," *Science*, vol. 359, no. 6374, pp. 418–424, 2018.

[5] N. Brown and T. Sandholm, "Superhuman AI for multiplayer poker," *Science*, vol. 365, no. 6456, pp. 885–890, 2019.

[6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[7] J. von Neumann and O. Morgenstern, *Theory of Games and Economic Behavior*. Princeton University Press, 1944.

[8] B. Chen and J. Ankenman, *The Mathematics of Poker*. ConJelCo, 2006.

[9] Python Software Foundation, "Python 3 documentation," 2024.

[10] Streamlit, "Streamlit documentation," 2024.

[11] W. McKinney, "Data structures for statistical computing in Python," in *Proc. Python in Science Conference*, 2010, pp. 56–61.

[12] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[13] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[14] OpenAI, "ChatGPT, AI-assisted writing and formatting tool, 2026.